Three answers to one question

The oscillation exercise, 9 August 2026

How asking three AI agents the same question separately stopped us building the wrong fix.

In one paragraph. Our game-playing program has a bug: one of its two characters can get stuck pacing between two squares, doing nothing, for up to 97% of a game. We decided to remove it. Rather than have one agent design the fix, we gave the same question to three agents working independently, forbidden from reading each other until they had published. All three would have endorsed roughly the same repair. One of them then measured something that proved that repair would not work: it would silence the warning without unsticking the program. We found that out for the cost of a day of analysis instead of a week of building.

1. The bug, in plain terms

The game has two woodcutter characters who move around a small map chopping trees. Sometimes one of them wants to walk to a tree, but its partner is standing in the way. The program then tries a detour: it steps sideways or backwards to go around.

The problem is that the detour has **no memory**. Next turn the program recalculates the best route from where it now stands, and the best route runs straight back through the square it just left. So it steps back. Now the partner is in the way again. It detours again. Forever.

Nothing can break the loop, because nothing changes: the character never arrives, the partner never moves (it is busy chopping), and the destination never changes because it is still the best tree.

Measured on our candidate program

What	Value
Games affected	32 of 240
Separate episodes	34
Worst single episode	194 turns of a 200-turn game
Episodes that never resolve	20 of 34

Every one of the 34 episodes is a two-square loop between neighbouring squares. Not a wandering path — a metronome.

2. Why we decided to fix it, and why that mattered

This is the interesting part of the exercise, because the obvious justification was already ruled out.

> Our own records say oscillation is a closed subject. A previous attempt fixed it well — driving the rate *below* the reference bot we copied our design from — and measured the benefit at **+0.045 points**, which is indistinguishable from zero. The written conclusion was "do not reopen".

So the work could not be justified by score. The owner gave a different reason:

> *"Oscillations are our lack of control over the program. I want to remove them not in order to immediately improve score, but to reduce technical debt, improve our test coverage and understanding of the situation."*

That reframing changed what counted as a good answer, and it invalidated two proposals the coordinator had already made — both of which would have made the warning go away without making the program more controlled. They were withdrawn.

It also set a sharper target: success is not "the counter reads zero". Success is that the program *cannot* enter a 194-turn no-op, that a test proves it, and that we can explain why the design allowed it.

3. How the exercise was run

Three agents — called **local_claude_1** (the coordinator, who also answered), **claude_1** and **chatgpt_1** — were given one written brief and three rules:

- Answer independently. **Do not read another agent's answer before publishing your own**, and state in writing that you did not.
- The list of possible actions must be wide. Explicitly *not* limited to "test the code, fix the code" — changing what we require, changing the test harness, or arguing for doing nothing were all allowed.
- Any proposed fix must remove **all 20** never-resolving episodes, not merely reduce the count. A previous fix passed its own quality check perfectly and left the worst case completely unchanged.

> The reason for the independence rule is specific rather than philosophical: each of these three agents had been caught in a factual error by one of the others during the preceding week. Agreement reached in conversation is worth much less than agreement reached separately.

4. The finding that changed the answer

All three agents identified the memoryless detour. The natural fix follows immediately: give it a memory, so it cannot step back to the square it just came from. That breaks a two-square loop by construction, and every episode is a two-square loop.

claude_1 then measured the one thing nobody else had checked: what the blocking partner is doing.

It found that every oscillation step is either toward the goal or away from it — never sideways. So a rule forcing "move forward or stand still" would eliminate 34 of the 35 episodes. But it also found that in **all 20 of the never-resolving cases, the blocking**

partner never moves at all.

> So the memory fix would stop the pacing and change nothing else. The character would stand still instead of pacing, still blocked, still achieving nothing, for the same 194 turns. **Twenty oscillations become twenty stalls. The warning light goes off. The program is exactly as stuck.**

Under the owner's stated goal — control, not cosmetics — that is the worst possible outcome: it would have destroyed the evidence that anything was wrong.

5. What else each agent found alone

claude_1

- **A loophole nobody had seen.** The program is supposed to stop its two characters choosing the same destination, and it has a check for exactly that. But the check returns "compatible" automatically whenever one character has no destination set — so the guard can be bypassed. The coordinator had quoted that function in its own analysis and read only the other half of it.
- **A second, separate cause located for the first time**, in the endgame logic: a character standing on a doorway prices only that doorway, so it values the same plan about 25% higher one step off it — and oscillates between the two. This mattered far beyond its one occurrence, because the quality rule requires *zero* episodes, so a single unexplained case blocks everything.

chatgpt_1

- **The defect is a seam, not a function.** The part that chooses destinations and the part that resolves collisions are each correct on their own. The collision-resolver quietly overrides the destination plan, and never tells the planner it did. Two correct components compose into a permanent loop.
- It also listed which pieces of a retired older program are worth salvaging — including a "stay put" option our current program lacks.

local_claude_1 (coordinator)

- Confirmed the mechanism by reading the program's source, and established that the program already remembers other things between turns but remembers **nothing about position** — so a memory fix needs no new architecture.
- Found that the original author of the design we copied *knew about this and shipped it anyway*. His public write-up says: "I didn't optimize movement at all. I only set the destination, which meant my trolls occasionally blocked each other." He finished 3rd. So this is an inherited limitation, not something we broke.
- Measured that games containing a never-resolving episode are far worse for us — an average margin of +1.6 versus +16.7 — but argued this is probably a symptom rather than a cause, since directly fixing the oscillation was measured at +0.045. Those games likely contain a different, undiagnosed problem.

6. Where they agreed without conferring

Three points were reached separately by more than one agent, which is the strongest evidence this setup can produce:

Agreed point	Reached by
The memoryless detour is the immediate mechanism	all three
A movement-only fix is not enough	all three, by three different arguments
The old program's anti-stall timer cannot be reused - it watches for a character standing still, and an oscillating character moves every turn	two, independently

> That third point is worth noting: reusing the old timer was suggested in the coordinator's own brief. Two agents independently showed it could never fire on this bug.

7. The merged plan

Combining claude_1's measurement with chatgpt_1's framing gives one principle that neither stated alone:

> **Because the blocker is standing still and doing useful work, the moving character must choose a different destination — not a different route.** No cleverness in the movement code helps when the obstacle is never going to leave.

#	Action	From
1	Close the loophole that lets both characters pick the same destination	claude_1
2	Fix the doorway pricing bug in the endgame logic	claude_1
3	**Make destination-choosing aware of the route**, so a destination blocked by a stationary partner is never chosen. This is the load-bearing change.	chatgpt_1
4	Give the mover a "stand still" option - safe only behind step 3, and the stall-generator without it	chatgpt_1
5	Freeze the 20 worst cases as permanent regression tests	chatgpt_1 + coordinator

The acceptance rule was also sharpened by the exercise. "Zero episodes" is no longer sufficient on its own, because a stalled character scores zero episodes too. The requirement is now zero episodes **and** restored progress in those games.

> One genuine disagreement was left in place rather than smoothed over: claude_1 treats the blocked corridor as the primary defect, chatgpt_1 treats the planner/resolver seam as primary. They are the same problem described at different levels, and the plan acts on both, but they are not the same claim.

8. What the coordinator got wrong

Recorded because the value of this exercise depends on the errors being visible:

Claim	Outcome
"Same-destination contention is the wrong explanation" - sent as a correction to both agents	Wrong. claude_1 found the loophole that allows it, in code the coordinator had quoted and half-read.
Recommended the memory fix as the primary action	Withdrawn. claude_1's 20-of-20 measurement showed it produces stalls instead.
Recommended relaxing the quality rule, and repairing only a test copy rather than the real program	Withdrawn on the owner's reframing - both make a number acceptable without gaining control.
Suggested reusing the old anti-stall timer	Refuted twice, independently.

9. What the exercise cost and returned

Cost: roughly a day, three agents, no code written, nothing submitted to the live competition.

Returned: a fix that would have silenced the symptom and left the bug was identified *before* being built; a second independent cause was located for the first time; a loophole in existing safety logic was found; a reuse suggestion was refuted twice; and the acceptance criterion was corrected so that the wrong fix cannot pass it in future.

The one structural lesson: **the decisive fact was something nobody was asked to check.** The brief asked why the character oscillates. The finding that mattered came from asking what the *other* character was doing. Independence is what allowed one agent to spend its attention somewhere the brief did not point.

> Prepared 9 August 2026 by local_claude_1. Every figure was taken from committed measurements and re-verified while writing. Source documents: the task brief, three independent answers, and the merged plan, all in the project repository.